

CM03 - C Introduction, Structure, Tools

C motivation, multi-file programs, compilation, make, debugging

Louis Ledoux

2026-07-05

Why C

Why C

Live pacing: about 8 min

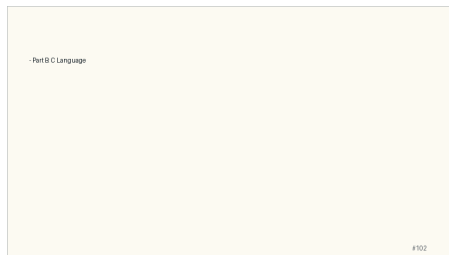
Question. Why learn C when higher-level languages exist?

Core claim. C exposes representation, memory, compilation, and linking decisions that other languages hide.

Target capability. Students can distinguish declarations, definitions, source files, headers, and object files.

Main Path

- ▶ Start from the question: Why learn C when higher-level languages exist?
- ▶ Build the model: C exposes representation, memory, compilation, and linking decisions that other languages hide.
- ▶ End with a checkable action: Students can distinguish declarations, definitions, source files, headers, and object files.
- ▶ Keep only this slide on the horizontal path during the live lecture.



Worked Example

- ▶ Use this as the live trace: Split a tiny counter program into `.h`, `.c`, and `main.c`, then break one declaration deliberately.
- ▶ Representative source slide: 106
- ▶ Ask students to predict the next state before revealing the explanation.
- ▶ Then connect the result back to the core claim.

What this course covers?

It is NOT a yet another programming course with extended presentation of the language syntax.
Differences with JAVA, similar syntaxes but still very different !
Mainly "tricky" parts of C language !
Being a master of C force is long and difficult
Requires practice, practice and practice
But, there is a lot of documentation !
The functions of libraries are documented
Example: `main {printf}`
Google, ChatGPT etc are your best friend...not to copy - paste, but read and understand !

#106

Anecdote Or Motivation

C is close enough to the machine that small textual changes can change storage, calling, or linkage.

- ▶ Use this only if students need a reason to care.
- ▶ If the room is already following, skip down to the check question.

Check Question

- ▶ Ask for a prediction, not a definition.
- ▶ Require a short justification: command trace, memory drawing, type rule, or file state.
- ▶ If more than a third of the room hesitates, go down to the source backup.

C language

- Developed by Thompson & Ritchie in 1972
- Originally developed 90-95% of the Unix kernel
- It has been standardized by the American National Standards Institute (ANSI) in 1988 (C ANSI)
- Loosely/weakly typed language (so warnings are important!)



#107

Backup Index

Original slides 102-107

- Part B: C Language

Backup: Part B: C Language

Original slide 102

- Part B: C Language

Backup: Outline

Original slide 103

- ▶ Introduction
- ▶ Structure of C program and compilation
- ▶ Basic components of C language
- ▶ Pointers and Arrays
- ▶ Functions and parameters
- ▶ Dynamic memory allocation
- ▶ Structures, Lists and Hash tables
- ▶ Strings
- ▶ I/O

Outline
- Introduction - Structure of C program and compilation - Basic components of C language - Pointers and Arrays - Functions and parameters - Dynamic memory allocation - Structures, Lists and Hash tables - Strings - I/O

#103

Backup: CM 3 Introduction, Structure and Tools

Original slide 104

-CM 3 Introduction, Structure and Tools

Backup: Why C ?

Original slide 105

JAVA is an evolved language
High abstraction away from the physical objects
Useful when we want to be abstracted from the hardware
Software engineering
C is closer the hardware
Relatively controls what happens to the memory
Manipulation of the physical objects of the hardware
System engineering
Main operating systems are written in C
Unix, GNU/Linux, etc.

WhyC?

- JAVA is an evolved language
- High abstraction away from the physical objects
- Useful when we want to be abstracted from the hardware
- Software engineering
- C is closer the hardware
- Relatively controls what happens to the memory
- Manipulation of the physical objects of the hardware
- System engineering
- Main operating systems are written in C
- Unix, GNU/Linux, etc.

#105

Backup: What this course covers?

Original slide 106

It is NOT a yet another programming course with extended

Differences with JAVA, similar syntaxes but still very different!

Mainly "tricky" parts of C language !

Being a master of C force is long and difficult

Requires practise, practise and practise

But, there is a lot of documentation !

The functions of libraries are documented

Example : man fprintf

Google, ChatGPT etc are your best friend...not to copy - paste, but read and

What this course covers?

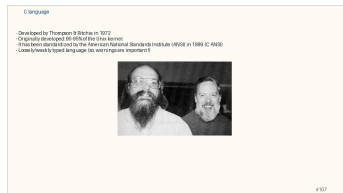
It is NOT a yet another programming course with extended presentation of the language syntax.
Differences with JAVA, similar syntaxes but still very different!
Being "tricky" master of C language!
Being a master of C force is long and difficult
Requires practise, practise and practise
But, there is a lot of documentation !
The functions of libraries are documented
Example : man fprintf
Google, ChatGPT etc are your best friend...not to copy - paste, but read and understand!

#106

Backup: C language

Original slide 107

- ▶ Developed by Thompson & Ritchie in 1972
- ▶ Originally developed: 90-95% of the Unix kernel
- ▶ It has been standardized by the American National Standards Institute (ANSI) in 1989 (C ANSI)
- ▶ Loosely/weakly typed language (so, warnings are important !)



Program Structure

Program Structure

Live pacing: about 9 min

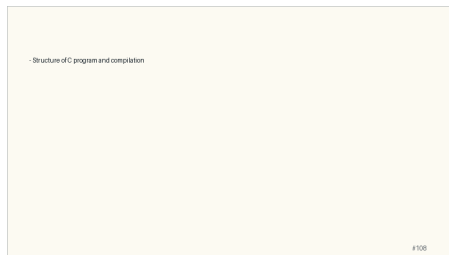
Question. Why learn C when higher-level languages exist?

Core claim. C exposes representation, memory, compilation, and linking decisions that other languages hide.

Target capability. Students can distinguish declarations, definitions, source files, headers, and object files.

Main Path

- ▶ Start from the question: Why learn C when higher-level languages exist?
- ▶ Build the model: C exposes representation, memory, compilation, and linking decisions that other languages hide.
- ▶ End with a checkable action: Students can distinguish declarations, definitions, source files, headers, and object files.
- ▶ Keep only this slide on the horizontal path during the live lecture.



Worked Example

- ▶ Use this as the live trace: Split a tiny counter program into `.h`, `.c`, and `main.c`, then break one declaration deliberately.
- ▶ Representative source slide: 113
- ▶ Ask students to predict the next state before revealing the explanation.
- ▶ Then connect the result back to the core claim.

Communication among .c files Example	
<pre>#include "code1.h" /* Declaration made available here */ /* Variable defined and initialized here */ int global_variable = 1; /* Definition checked against declaration */ int increment(void) { return global_variable + 1; }</pre>	-code1.c
<pre>extern int global_variable; /* Declaration of the variable */</pre>	-code1.h
<pre>#include "code1.h" #include <stdio.h> void use_it(void) { printf("Global variable: %d\n", global_variable); }</pre>	-code2.c
	#113

Anecdote Or Motivation

C is close enough to the machine that small textual changes can change storage, calling, or linkage.

- ▶ Use this only if students need a reason to care.
- ▶ If the room is already following, skip down to the check question.

Check Question

- ▶ Ask for a prediction, not a definition.
- ▶ Require a short justification: command trace, memory drawing, type rule, or file state.
- ▶ If more than a third of the room hesitates, go down to the source backup.

Some example split in 3 files

```
/*=====
 * main.c
 *=====*/

#include <stdlib.h>
#include <stdio.h>
#include "ppcm.h"
#define PI 3.14

int variableGloble = 3;
//extern int ppcm (int a, int b);

void printINT(int val)
{
    printf("%d\n",val);
}

int main(int argc, char *argv[])
{
    int variableLocale = ppcm(2,variableGloble);
    printINT(variableLocale);
    return EXIT_SUCCESS;
}
```

```
/*=====
 * ppcm.h
 *=====*/

#ifndef PPCM_H
#define PPCM_H

int ppcm(int x, int y);

#endif

/*=====
 * ppcm.c
 *=====*/
int ppcm(int x, int y)
{
    int a=x;
    int b=y;
    while (a!=b)
    {
        while (a>b) b=b+y;
        while (a<b) a=a-x;
    }
    return a;
}
```

#120

Backup Index

Original slides 108-120

- Structure of C program and compilation

Backup: Structure of C program and compilation

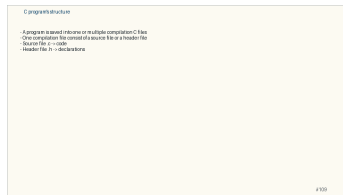
Original slide 108

- Structure of C program and compilation

Backup: C program's structure

Original slide 109

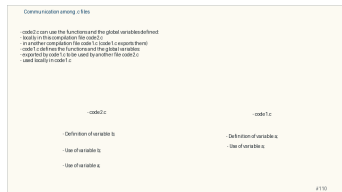
- ▶ A program is saved into one or multiple compilation C files
- ▶ One compilation file consist of a source file or a header file
- ▶ Source file .c -> code
- ▶ Header file .h -> declarations



Backup: Communication among .c files

Original slide 110

- ▶ code2.c can use the functions and the global variables defined:
 - ▶ locally in this compilation file code2.c
 - ▶ in another compilation file code1.c (code1.c exports them)
- ▶ code1.c defines the functions and the global variables:
 - ▶ exported by code1.c to be used by another file code2.c
 - ▶ used locally in code1.c
- ▶ code2.c
- ▶ code1.c
- ▶ Definition of variable b;
- ▶ Definition of variable a;
- ▶ Use of variable a;
- ▶ Use of variable b;
- ▶ Use of variable a;



Backup: Communication among .c files

Original slide 111

The definition of the global variables and constants defined in code1.c -> "seen" only by code1.c

To be "seen" by file code2.c -> the global variable declarations are placed in the header file code1.h

extern: For the global variable to be seen by other definitions are in the source file code1.c

code2.c includes the header code1.h of code1.c through the directive #include

Communication among .c files

The definition of the global variables and constants is by default local, defined in code1.c -> "seen" only by code1.c.
To be "seen" by file code2.c -> the global variable has to be exported, declarations to be placed in the header file code1.h.
extern: For the global variables to be seen by other .c files, definitions are in the source file code1.c.
code2.c includes the header code1.h of code1.c through the directive #include

#111

Backup: Directive #include

Original slide 112

Syntax:

#include <filename.h> (1)

#include "filename.h" (2)

Description

(1) and (2): Include the file called filename which is located in the directory /usr/include

Only (2): Search the file called filename in the working directory if the file is saved in another directory you have to give the path "path/filename"

Functionality:

Includes (before compilation) the content of filename. This text is treated as if it was part of the source file.



Backup: Communication among .c files: Example

Original slide 113

```
#include "code1.h" /* Declaration made available here */  
/* Variable defined and initialized here */  
int global_variable = 1; /* Definition checked against declaration */  
int increment(void) { return global_variable++; }
```

- ▶ code1.c
- ▶ extern int global_variable; /* Declaration of the variable */
- ▶ code1.h

```
#include "code1.h"  
#include <stdio.h>  
void use_it(void)  
{  
    printf("Global variable: %d\n", global_variable);  
}
```

Communication among .c files Example	
<pre>#include "code1.h" /* Declaration made available here */ /* Variable defined and initialized here */ int global_variable = 1; /* Definition checked against declaration */ int increment(void) { return global_variable++; }</pre>	code1.c
<pre>extern int global_variable; /* Declaration of the variable */</pre>	code1.h
<pre>#include "code1.h" #include <stdio.h> void use_it(void) { printf("Global variable: %d\n", global_variable); }</pre>	code2.c
#113	

Backup: Communication among .c files: Example

Original slide 114

- ▶ `extern void use_it(void);`
- ▶ `extern int increment(void);`
- ▶ `prog.h`

```
#include "code1.h"
#include "prog.h"
#include <stdio.h>
int main(void)
{
    use_it();
    global_variable += 10;
    use_it();
    increment();
    use_it();
    return 0;
}
```

Communication among .c files Example	
<code>-extern void use_it(void);</code> <code>-extern int increment(void);</code>	<code>- prog.h</code>
<code>#include "code1.h"</code> <code>#include "prog.h"</code> <code>#include <stdio.h></code> <code>int main(void)</code> <code>{</code> <code> use_it;</code> <code> global_variable += 10;</code> <code> use_it;</code> <code> increment();</code> <code> use_it;</code> <code> return 0;</code> <code>}</code>	<code>- prog.c</code>
#114	

Backup: Communication among .c files: Example

Original slide 115

```
#include "code1.h" /* Declaration made available here */
#include "prog.h"
/* Variable defined here */
int global_variable = 1; /* Definition checked against declaration */
int increment(void) { return global_variable++; }
```

- ▶ code1.c
- ▶ extern int global_variable; /* Declaration of the variable */
- ▶ code1.h

```
#include "code1.h"
#include "prog.h"
#include <stdio.h>
void use_it(void)
{
    printf("Global variable: %d\n", global_variable);
}
```

Communication among .c files Example	
#include "code1.h" /* Declaration made available here */ #include "prog.h" /* Variable defined here */ int global_variable = 1; /* Definition checked against declaration */ int increment(void) { return global_variable++; }	code1.c
extern int global_variable; /* Declaration of the variable */	code1.h
#include "code1.h" #include "prog.h" #include <stdio.h> void use_it(void) { printf("Global variable: %d\n", global_variable); }	code2.c
	#115

Backup: General rules

Original slide 116

- ▶ For any given global variable
- ▶ exactly 1 header file declares it
- ▶ exactly 1 source file defines it and preferably initializes it too.
- ▶ (otherwise undefined behavior -> anything could happen !)
- ▶ The source file that defines the variable also includes the header (ensuring definition and declaration are consistent)
- ▶ A source file
- ▶ never contains extern declarations of variables
- ▶ always include the (sole) header that declares them
- ▶ A header file only contains extern declarations of variables
- ▶ not static
- ▶ (each source file makes its own version of the global

General rules

- For any given global variable
- exactly 1 header file declares it
- exactly 1 source file defines it and preferably initializes it too
- otherwise undefined behavior -> anything could happen !
- The source file that defines the variable also includes the header (ensuring definition and declaration are consistent)
- A source file
- never contains extern declarations of variables
- always include the (sole) header that declares them
- A header file only contains extern declarations of variables
- not static
- each source file makes its own version of the global variable

#190

Backup: Structure of .c file

Original slide 117

- ▶ Description of the functionality and the use (comments)
- ▶ The directives for the pre-processor
- ▶ Include files, Macros, Conditional compilation, ...
- ▶ Global variables used only by this file
- ▶ if not initialized, they are by default 0
- ▶ The functions that act on the variables
- ▶ Function prototype
- ▶ Function body
- ▶ No order is imposed to the definitions, BUT any element has to be declared BEFORE its utilisation

Structure of a file

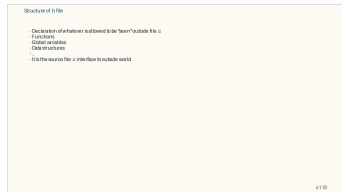
- Description of the functionality and the use (comments)
- The directives for the pre-processor
- Include files, Macros, Conditional compilation, ...
- Global variables used only by this file
- if not initialized, they are by default 0
- The functions that act on the variables
- Function prototype
- Function body
- No order is imposed to the definitions, BUT any element has to be declared BEFORE its utilisation

#117

Backup: Structure of .h file

Original slide 118

- ▶ Declaration of whatever is allowed to be "seen" outside file .c
- ▶ Functions
- ▶ Global variables
- ▶ Data structures
- ▶ ...
- ▶ It is the source file .c interface to outside world



Backup: Structure of .c file: Example

Original slide 119

- ▶ Pre-processor directives
 - ▶ Include files
 - ▶ Macros
- ▶ Global variable defined in this file
- ▶ Function declaration: is defined in another file
- ▶ Function defined in this file (can be exported)
- ▶ The main function is the starting point
- ▶ Only 1 main function!

Structure of a file Example

```
/*=====
 * ppon.c
 *=====*/
int ppon(int x, int y)
{
    int ans;
    int temp;
    while (x!=0)
    {
        while (y!=0) temp++;
        while (x!=0) ans++;
    }
    return ans;
}
```

```
/*=====
 * main.c
 *=====*/
#include <stdio.h>
#include <stdlib.h>

#define PI 3.14

int variableGloble = 3;

extern int ppon (int a, int b);

void printINT(int val)
{
    printf("%d\n",val);
}

int main(int argc, char *argv[])
{
    int variableLocale = ppon(2,variableGloble);
    printINT(variableLocale);
    return EXIT_SUCCESS;
}
```

- Pre processor directives
- Include files
- Macros

- Global variable defined in this file

- Function-declaration: is defined in another file

- Function defined in this file
can be exported

- The main function is the
starting point
- Only 1 main function!

#119

Backup: Same example split in 3 files

Original slide 120

Same example split in 3 files

```
/*  
 * main.c  
 */  
  
#include <stdlib.h>  
#include <stdio.h>  
  
#include "ppcm.h"  
  
#define PI 3.14  
  
int variableGlobale = 3;  
  
//extern int ppcm (int a, int b);  
  
void printINT(int val)  
{  
    printf("%d\n",val);  
}  
  
int main(int argc, char *argv[])  
{  
    int variableLocale = ppcm(2,variableGlobale);  
    printINT(variableLocale);  
    return EXIT_SUCCESS;  
}
```

```
/*  
 * ppcm.h  
 */  
  
#ifndef PPCM_H  
#define PPCM_H  
  
int ppcm(int x, int y);  
  
#endif  
  
/*  
 * ppcm.c  
 */  
int ppcm(int x, int y)  
{  
    int a=x;  
    int b=y;  
    while (a!=b)  
    {  
        while (a>b) b=b+y;  
        while (a<b) a=a+x;  
    }  
    return a;  
}
```

Compilation

Compilation

Live pacing: about 8 min

Question. What happens before an executable exists?

Core claim. Compilation is a pipeline; make records the dependency reasoning humans otherwise repeat by hand.

Target capability. Students can identify preprocess, compile, link, warning, and rebuild steps.

Main Path

- ▶ Start from the question: What happens before an executable exists?
- ▶ Build the model: Compilation is a pipeline; make records the dependency reasoning humans otherwise repeat by hand.
- ▶ End with a checkable action: Students can identify preprocess, compile, link, warning, and rebuild steps.
- ▶ Keep only this slide on the horizontal path during the live lecture.

JAVA compilation

- The .java files have the code that will be translated into the files .class by the compiler `javac`
- The program is split into classes
- The files .class consist of the bytecode (java a lower level language closer to the machine language (but not a machine language))
- The bytecode can be executed in whatever machine that has a Java Virtual Machine. The JVM interprets the bytecode to machine language (portable, but slow)
- The link among classes is performed during the execution (Dynamic link)

#121

Worked Example

- ▶ Use this as the live trace: Show the same program built manually, then with a Makefile, then after editing one header.
- ▶ Representative source slide: 126
- ▶ Ask students to predict the next state before revealing the explanation.
- ▶ Then connect the result back to the core claim.

C compilation: GCC

- GNU (GNU is Not Unix) Compiler Collection is free software.
- Design to target several machines
- Exist by default in Linux
- Syntax (see man gcc):
 - gcc [-c] [-g] [-O] [-f] ...
- Options
 - c: Produce an object file o from c file by stopping before linking
 - i-dir: add dir in the include path (used header files, h)
 - o nom: Specifies the name of output file. By default, object files have the same name as their code files

#126

Anecdote Or Motivation

A good Makefile is a memory aid for the team; it documents what must be rebuilt and why.

- ▶ Use this only if students need a reason to care.
- ▶ If the room is already following, skip down to the check question.

Check Question

- ▶ Ask for a prediction, not a definition.
- ▶ Require a short justification: command trace, memory drawing, type rule, or file state.
- ▶ If more than a third of the room hesitates, go down to the source backup.

GCC: Several compilation files

- Produce an object file called module.o from a code file module.c (-c stop before linking)
- gcc -o module.o -c module.c
- Produce an executable file named prog by linking its object files using -o
- gcc -o prog module1.o ... moduleN.o

#1.28

Backup Index

Original slides 121-128

JAVA compilation

- The .java files have the code that will be translated into the files.class by the compiler
javac
- The program is split into classes
- The files.class consist of the bytecode java: a lower level language closer to the machine language (but not a machine language)
- The bytecode can be executed in whatever machine that has a Java Virtual Machine. The JVM interprets the bytecode to machine language (portable, but slow)
- The link among classes is performed during the execution (Dynamic link)

Backup: JAVA compilation

Original slide 121

- ▶ The .java files have the code that will be translated into the files .class by the compiler javac
- ▶ The program is split into classes
- ▶ The files.class consist of the bytecode java: a lower level language closer to the machine language (but not a machine language)
- ▶ The bytecode can be executed in whatever machine that has a Java Virtual Machine. The JVM interprets the bytecode to machine language (portable, but slow)
- ▶ The link among classes is performed during the execution (Dynamic link)



Backup: JAVA compilation

Original slide 122

- ▶ source Java
- ▶ Libraries (.jar)
- ▶ Java compiler
- ▶ File .class
- ▶ (bytecode)
- ▶ JAVA Virtual
- ▶ Machine
- ▶ File .class
- ▶ (bytecode)
- ▶ Execution
- ▶ File .class
- ▶ (bytecode)



Backup: C compilation

Original slide 123

The call of the C compiler is associated with 3 tools

1st step: The pre-processor (CPP):

Handles the directives (#include, #define etc.)

2nd step: C compiler and assembler (GCC):

C compiler performs one-to-one translation of a `.c` file

This step is hidden by default (use `gcc -S` to see it)

Then, the assembler generates an object file (`.o`) with

the machine instructions of the functions defined in this `.c` file

information on the functions used in this `.c` file, but defined in another file

3rd step: The linker (LD/GCC):

Makes the link (resolves symbols) among the different program's **objects** (static

C compilation

This call of the C compiler is associated with 3 tools

- 1st step: The pre-processor (CPP):
Handles the directives (#include, #define etc.)
- 2nd step: C compiler and assembler (GCC):
C compiler performs one-to-one translation of a `.c` file into an assembly file (`.s`)
- 3rd step: The linker (LD/GCC):
Then, the assembler generates an object file (`.o`) with the machine instructions of the functions defined in this `.c` file, but defined in another file

#123

Backup: C compilation

Original slide 124

- ▶ The generated binary depends on
- ▶ Architecture/processor
- ▶ AND
- ▶ operating system
- ▶ -> It is not portable !
- ▶ Possible to use "cross compilation"
- ▶ Use a machine to create an binary for another machine
- ▶ -march
- ▶ native
- ▶ i386

C compilation

- The generated binary depends on
 - architecture/processor
 - AND
 - operating system
 - > It is not portable !
- Possible to use "cross compilation"
- Use a machine to create an binary for another machine
- march
- native
- i386

#124

Backup: C compilation

Original slide 125

- ▶ User C source file (.c)
- ▶ ▶ headers (.h)
- ▶ Dynamic Library
- ▶ (ex : .dll, .so)
- ▶ Preprocessor +
- ▶ C compiler
- ▶ Program in machine
- ▶ language stored in RAM
- ▶ Loader (OS)
- ▶ Assembly source
- ▶ Assembler
- ▶ Executable file
- ▶ File objet1 (.o)
- ▶ Linker
- ▶ File objet2 (.o)
- ▶ Execution



Backup: C compilation: GCC

Original slide 126

- ▶ GNU (GNU is Not Unix!) Compiler Collection is free software:
- ▶ Design to target several machines
- ▶ Exist by default in Linux
- ▶ Syntax (see `man gcc`):
- ▶ `gcc [-c] [-g] [-I dir] [-o nom] f1 ... fn`
- ▶ Options
- ▶ `-c` : Produce an object file `.o` from `.c` file by stopping before linking
- ▶ `-I dir` : adds `dir` in the include path (used header files `.h`)
- ▶ `-o nom` : Specifies the name of output file. By default:
- ▶ object files have the same name as their code files
- ▶ Executable file: `a.out`
- ▶ `-g` : Inserts the information needed by the debugger



Backup: GCC: One compilation file

Original slide 127

- ▶ Produce an executable called module from the file module.c
- ▶ gcc module.c -o module

GCC: One compilation file

Produce an executable called module from the file module.c
gcc module.c -o module

```
/*****  
 * main.c  
 * la classique Hello World  
 *****/  
  
#include <stdio.h>  
#include <stdlib.h>  
  
int main(void)  
{  
    printf("Hello World\n");  
    return EXIT_SUCCESS;  
}
```

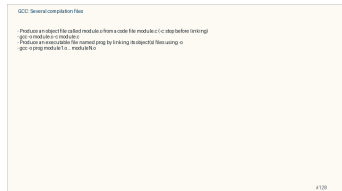
```
bash-3.2$ ls  
main.c  
bash-3.2$ gcc main.c -o hello  
bash-3.2$ ls  
hello  main.c  
bash-3.2$ ./hello  
Hello World  
bash-3.2$ █
```

#127

Backup: GCC: Several compilation files

Original slide 128

- ▶ Produce an object file called module.o from a code file module.c (-c: stop before linking)
- ▶ `gcc -o module.o -c module.c`
- ▶ Produce an executable file named prog by linking its object(s) files using -o
- ▶ `gcc -o prog module1.o ... moduleN.o`



Make

Make

Live pacing: about 8 min

Question. What happens before an executable exists?

Core claim. Compilation is a pipeline; `make` records the dependency reasoning humans otherwise repeat by hand.

Target capability. Students can identify preprocess, compile, link, warning, and rebuild steps.

Main Path

- ▶ Start from the question: What happens before an executable exists?
- ▶ Build the model: Compilation is a pipeline; make records the dependency reasoning humans otherwise repeat by hand.
- ▶ End with a checkable action: Students can identify preprocess, compile, link, warning, and rebuild steps.
- ▶ Keep only this slide on the horizontal path during the live lecture.

C compilation: The tool make

Principle

- Use to automate the compilation (large) projects, which consists of several files
- Handles file dependencies (e.g. #include)
- Allow to automatize the update of the files

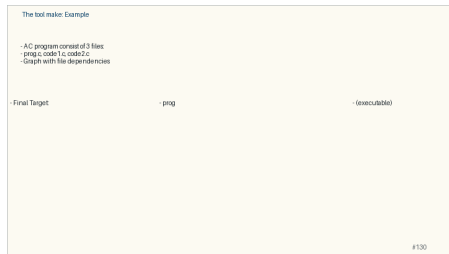
Functionality: Create a file called Makefile that

- Defines the dependencies between targets
- Defines the dependencies between each target and the corresponding files
- Provides the commands required to be executed to create the target based on the dependencies (flags)

#129

Worked Example

- ▶ Use this as the live trace: Show the same program built manually, then with a Makefile, then after editing one header.
- ▶ Representative source slide: 130
- ▶ Ask students to predict the next state before revealing the explanation.
- ▶ Then connect the result back to the core claim.



Anecdote Or Motivation

A good Makefile is a memory aid for the team; it documents what must be rebuilt and why.

- ▶ Use this only if students need a reason to care.
- ▶ If the room is already following, skip down to the check question.

Check Question

- ▶ Ask for a prediction, not a definition.
- ▶ Require a short justification: command trace, memory drawing, type rule, or file state.
- ▶ If more than a third of the room hesitates, go down to the source backup.

Structure of the Makefile: Example 2

```
#####  
* hello.o  
#####  
#ifndef HELLO_H  
#define HELLO_H  
void Hello(void);  
#endif  
#####  
* hello.o  
#####  
#include <stdio.h>  
void Hello(void)  
{  
    printf("Hello World\n");  
}  
#####  
* main.o  
#####  
#include <stdlib.h>  
#include "hello.h"  
int main(void)  
{  
    Hello();  
    return EXIT_SUCCESS;  
}  
#####
```

```
CC=gcc  
CFLAGS=-g -Wall -I  
DEP=hello.h  
OBJ=hello.o  
EXEC=hello  
all: $(EXEC)  
$(EXEC): $(OBJ)  
$(CC) -o $@ $+  
%o %c $(DEP)  
$(CC) -c $(CFLAGS) $< -o $@  
PACONY: clean mproper  
clean:  
rm -f.o  
mproper: clean  
rm $(EXEC)  
#140
```

Backup Index

Original slides 129-140

C compilation: The tool make

Principle

Use to automate the compilation (large) projects, which consists of several files

Handles file dependencies (e.g. #includes)

Allow to automatize the update of the files

Functionality: Create a file called Makefile that

Defines the dependencies between targets

Defines the dependencies between each target and the corresponding files

Provides the commands required to be executed to create the target based on the dependencies (files)

Backup: C compilation: The tool make

Original slide 129

Principle

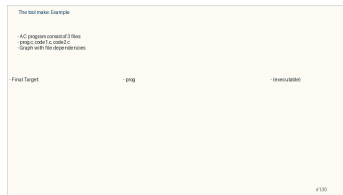
Use to automate the compilation (large) projects, v
Handles file dependencies (e.g. #includes)
Allow to automatize the update of the files
Functionality: Create a file called Makefile that
Defines the dependencies between targets
Defines the dependencies between each target and the corresponding files
Provides the commands required to be executed to create the target based on



Backup: The tool make: Example

Original slide 130

- ▶ A C program consist of 3 files:
- ▶ prog.c, code1.c, code2.c
- ▶ Graph with file dependencies
- ▶ prog
- ▶ Final Target:
- ▶ (executable)



Backup: The tool make: Example

Original slide 131

- ▶ A C program consist of 3 files:
- ▶ prog.c, code1.c, code2.c
- ▶ Graph with file dependencies
- ▶ prog
- ▶ Final Target:
- ▶ (executable)
- ▶ (object)
- ▶ code2.o
- ▶ code1.o
- ▶ prog.o
- ▶ To produce the executable file prog, the following command has to be executed :
- ▶ gcc -o prog prog.o code1.o code2.o
- ▶ The file prog depends on the object files prog.o, code1.o and code2.o



Backup: The tool make: Example

Original slide 132

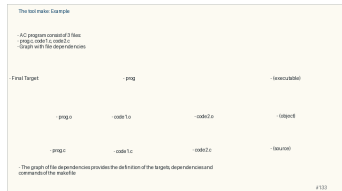
- ▶ A C program consist of 3 files:
- ▶ prog.c, code1.c, code2.c
- ▶ Graph with file dependencies
- ▶ prog
- ▶ Final Target:
- ▶ (executable)
- ▶ (object)
- ▶ code2.o
- ▶ code1.o
- ▶ prog.o
- ▶ (source)
- ▶ prog.c
- ▶ To produce the object file prog.o, we have to execute
- ▶ gcc -c prog.c -o prog.o
- ▶ So, the file prog.o depends on the source file prog.c



Backup: The tool make: Example

Original slide 133

- ▶ A C program consist of 3 files:
- ▶ prog.c, code1.c, code2.c
- ▶ Graph with file dependencies
- ▶ prog
- ▶ Final Target:
- ▶ (executable)
- ▶ (object)
- ▶ code2.o
- ▶ code1.o
- ▶ prog.o
- ▶ (source)
- ▶ code2.c
- ▶ prog.c
- ▶ code1.c
- ▶ The graph of file dependencies provides the definition of the targets, dependencies and commands of the



Backup: Structure of the Makefile

Original slide 134

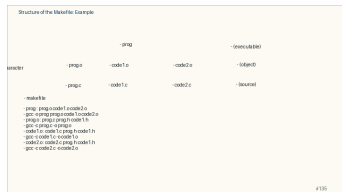
- ▶ Format of a rule
- ▶ target: dep_file1 dep_file2 ... (TAB) command
- ▶ Application of rule:
- ▶ If the date of the last modification in one of the dependent files dep_file1, dep_file2, etc. is more recent than the date of the file target, the tool make executes the command
- ▶ Use: command make
- ▶ The tool executes the first target of the file makefile of the working directory
- ▶ make label: Tool executes the rule associated with the target called label



Backup: Structure of the Makefile: Example

Original slide 135

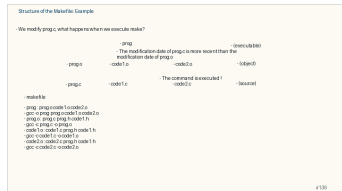
- ▶ prog
- ▶ (executable)
- ▶ (object)
- ▶ code2.o
- ▶ code1.o
- ▶ prog.o
- ▶ TAB character
- ▶ (source)
- ▶ code2.c
- ▶ code1.c
- ▶ prog.c
- ▶ makefile
- ▶ prog : prog.o code1.o code2.o
- ▶ gcc -o prog prog.o code1.o code2.o
- ▶ prog.o : prog.c prog.h code1.h
- ▶ gcc -c prog.c -o prog.o



Backup: Structure of the Makefile: Example

Original slide 136

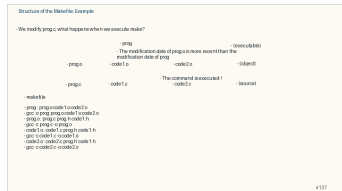
- ▶ We modify prog.c, what happens when we execute make?
- ▶ prog
- ▶ (executable)
- ▶ The modification date of prog.c is more recent than the modification date of prog.o
- ▶ (object)
- ▶ code2.o
- ▶ code1.o
- ▶ prog.o
- ▶ The command is executed !
- ▶ (source)
- ▶ code2.c
- ▶ code1.c
- ▶ prog.c
- ▶ makefile



Backup: Structure of the Makefile: Example

Original slide 137

- ▶ We modify prog.c, what happens when we execute make?
- ▶ prog
- ▶ (executable)
- ▶ The modification date of prog.o is more recent than the modification date of prog
- ▶ (object)
- ▶ code2.o
- ▶ code1.o
- ▶ prog.o
- ▶ The command is executed !
- ▶ (source)
- ▶ code2.c
- ▶ code1.c
- ▶ prog.c
- ▶ makefile



Backup: Structure of the Makefile: Example

Original slide 138

- ▶ Add a clean target: Remove object files, executable
- ▶ Use: make clean
- ▶ makefile

```
prog : prog.o code1.o code2.o
gcc -o prog prog.o code1.o code2.o
prog.o : prog.c prog.h code1.h
gcc -c prog.c -o prog.o
code1.o : code1.c prog.h code1.h
gcc -c code1.c -o code1.o
code2.o : code2.c prog.h code1.h
gcc -c code2.c -o code2.o
clean :
rm *.o prog
```

Structure of the Makefile: Example

- Add a clean target: Remove object files, executable
Use: make clean

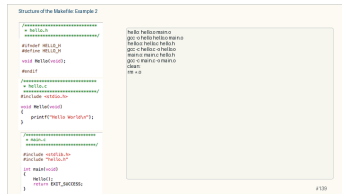
```
- makefile
prog: prog.o code1.o code2.o
gcc -o prog prog.o code1.o code2.o
prog.o: prog.c prog.h code1.h
gcc -c prog.c -o prog.o
code1.o: code1.c prog.h code1.h
gcc -c code1.c -o code1.o
code2.o: code2.c prog.h code1.h
gcc -c code2.c -o code2.o
clean:
rm *.o prog
```

#138

Backup: Structure of the Makefile: Example 2

Original slide 139

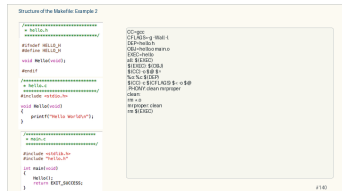
```
hello: hello.o main.o
gcc -o hello hello.o main.o
hello.o: hello.c hello.h
gcc -c hello.c -o hello.o
main.o: main.c hello.h
gcc -c main.c -o main.o
clean:
rm *.o
```



Backup: Structure of the Makefile: Example 2

Original slide 140

```
CC=gcc
CFLAGS=-g -Wall -I.
DEP=hello.h
OBJ=hello.o main.o
EXEC=hello
all: $(EXEC)
$(EXEC): $(OBJ)
$(CC) -o $@ $^
%.o: %.c $(DEP)
$(CC) -c $(CFLAGS) $< -o $@
.PHONY: clean mrproper
clean:
rm *.o
mrproper: clean
rm $(EXEC)
```



Diagnostics and Tools

Diagnostics and Tools

Live pacing: about 8 min

Question. What does the operating system do between the program and the machine?

Core claim. The OS gives a program controlled access to files, processes, memory, and devices.

Target capability. Students can place shell commands, processes, files, and permissions in the OS layer.

Main Path

- ▶ Start from the question: What does the operating system do between the program and the machine?
- ▶ Build the model: The OS gives a program controlled access to files, processes, memory, and devices.
- ▶ End with a checkable action: Students can place shell commands, processes, files, and permissions in the OS layer.
- ▶ Keep only this slide on the horizontal path during the live lecture.

Common compilation error messages

- Example of an error message
- Set of errors!
- Always start correcting the first one (they can be dependent!!)

```
bash-3.2$ gcc main.c ppcw.c -o main
main.c:13: error: expected '}', '(', '}', 'asm' or '___attribute__' before 'main'
bash-3.2$
```



```
bash-3.2$ gcc main.c ppcw.c -o main
main.c: In function 'main':
main.c:21: error: 'a' undeclared (first use in this function)
main.c:21: error: (Each undeclared identifier is reported only once
main.c:21: error: for each function it appears in.)
main.c:22: error: expected ';' before 'int'
main.c:23: error: 'variableLocale' undeclared (first use in this function)
bash-3.2$
```

#141

Worked Example

- ▶ Use this as the live trace: Compare `ls`, an editor, and a compiled program: all are processes asking the OS for services.
- ▶ Representative source slide: 146
- ▶ Ask students to predict the next state before revealing the explanation.
- ▶ Then connect the result back to the core claim.



Anecdote Or Motivation

A useful mental image: user programs are guests; the OS is the host that owns the keys.

- ▶ Use this only if students need a reason to care.
- ▶ If the room is already following, skip down to the check question.

Check Question

- ▶ Ask for a prediction, not a definition.
- ▶ Require a short justification: command trace, memory drawing, type rule, or file state.
- ▶ If more than a third of the room hesitates, go down to the source backup.

Code style is very important!

- Understanding the code written by someone else is a hard job!
- Use a universal language (readable by everyone)
- Decide a code style and be CONSISTENT
- Comments
- Documentation: Programmers that want to use the code
- Implementation: Programmers that maintain the code, i.e. they have to understand your implementation
- Names that mean something
- `int z % int counter`

#147

Backup Index

Original slides 141-147

Common compilation error messages

- Example of an error message
- Set of errors!
- Always start correcting the first one (they can be dependent)!!

```
bash-3.2$ gcc main.c ppcm.c -o main
main.c:19: error: expected '=', ',', ';', 'asm' or '__attribute__' before 'main'
bash-3.2$ █
```

```
bash-3.2$ gcc main.c ppcm.c -o main
main.c: In function 'main':
main.c:21: error: 'a' undeclared (first use in this function)
main.c:21: error: (Each undeclared identifier is reported only once
main.c:21: error: for each function it appears in.)
main.c:22: error: expected ';' before 'int'
main.c:23: error: 'variableLocale' undeclared (first use in this function)
bash-3.2$ █
```

Backup: Common compilation error messages

Original slide 141

- ▶ Example of an error message
- ▶ Set of errors!
- ▶ Always start correcting the first one (they can be dependent)!!

```
Common compilation error messages

- Example of an error message
- Set of errors!
- Always start correcting the first one (they can be dependent)!!

bash-3.2$ gcc main.c printf.c -o main
main.c:118: error: expected '}', ',', '}', '}', 'asm' or '___attribute__' before 'main'
bash-3.2$ █

bash-3.2$ gcc main.c printf.c -o main
main.c: In function 'main':
main.c:121: error: 'a' undeclared (first use in this function)
main.c:121: error: [Each undeclared identifier is reported only once]
main.c:121: error: for each function it appears in,
main.c:125: error: expected ';' before 'us'
main.c:125: error: 'variableScale' undeclared (first use in this function)
bash-3.2$ █
```

Backup: Debugging

Original slide 142

Usual bugs

Infinite loops

Loop boundaries

Non-Initialized variables

Segmentation faults

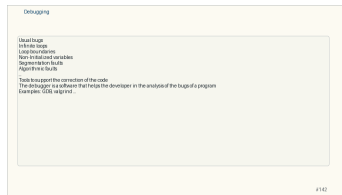
Algorithmic faults

...

Tools to support the correction of the code

The debugger is a software that helps the developer in the analysis of the bugs

Examples : GDB, valgrind ...



Backup: Tool GDB

Original slide 143

With the GDB, we can

- Start the execution of the program
- Stop the execution of the program based on some condition
- Examine what happened when the program stopped
- Execute the program step-by-step
- Perform modifications during the execution of the program
- Use: Make the link between the program and the debugger
- Add the information of debug during compilation
- Option `-g` and `-ggdb`



Backup: Tool GDB: Commands

Original slide 144

```
Start execution: run
Show the stack of the call: backtrace
Break Point : break (help breakpoints)
Continue execution : cont
Execute one instruction : step
Display the variables
Display nom_variable
watch nom_variable (allows to monitor)
info locals (local variables)
...
GDB Quick Reference Sheet !!!
```



Backup: Tool Valgrind

Original slide 145

With Valgrind we can identify memory leaks

Example of a memory leak

Use: `valgrind --tool=memcheck ./test`

Tool Valgrind

With Valgrind we can identify memory leaks
Example of a memory leak
Use: `valgrind --tool=memcheck ./test`

```
/*=====
 * test.c
 *=====*/
#include <stdio.h>
int main()
{
    int a=2;
    int* p = (int*) malloc(1024);
    printf("%i\n", *p);
    return 0;
}
```

#145

Backup: ...before going in more details about the C syntax...

Original slide 146

...before going in more details about the C syntax...

Backup: Code style is very important!

Original slide 147

- ▶ Understanding the code written by someone else is a hard job!
- ▶ Use a universal language (readable by everyone)
- ▶ Decide a code style and be CONSISTENT
- ▶ Comments
- ▶ Documentation: Programmers that want to use the code
- ▶ Implementation: Programmers that maintain the code, i.e. they have to understand your implementation
- ▶ Names that mean something
- ▶ `int z` Vs `int counter`

Code style is very important

- Understanding the code written by someone else is a hard job!
- Use a universal language (readable by everyone)
- Decide a code style and be CONSISTENT
- Comments
- Documentation: Programmers that want to use the code
- Implementation: Programmers that maintain the code, i.e. they have to understand your implementation
- Names that mean something
- `int z` Vs `int counter`

47/47